

Go2 Reinforcement Learning Locomotion and Sim-to-Real Transfer

Assignment 13 (E4): training in NVIDIA Isaac Lab, deployment on the real Unitree Go2

Richard Albert King Mechoda | ID 8525970

Cognitive Architectures for Robotics (COGAR), University of Genoa, DIBRIS

Make a real quadruped walk, without training on it

- Goal: a controller that makes the Unitree Go2 walk, trot and track velocity commands.
- Training on real legged hardware is slow, costly and damaging (thousands of falls, motor wear).
- Approach: learn the controller entirely in simulation, then deploy it zero-shot on the real robot.
- The research problem is the sim-to-real gap: a policy excellent in simulation can fail on hardware.
- Some signals the simulator gives for free, such as the body linear velocity, cannot be measured on the robot.



Project at a glance

A walking controller trained entirely in simulation and transferred to the real robot with no real-world fine-tuning.

2 billion

training timesteps

16,384

robots in parallel

1 h 40 m

training, one RTX PRO
6000

99.7 %

survival in simulation

0.22 m/s

velocity tracking error

Flat terrain: deployed zero-shot on the real Go2 (works straight and smooth).

Rough terrain (stairs, uneven ground, all directions): trained and validated in simulation; hardware deployment pending.

Four ways the field closes the sim-to-real gap

Simulator fidelity / system ID

Match the simulator to the robot: measured latency, learned actuator networks.

Tan 2018, Hwangbo 2019

Domain randomization

Train over a distribution of physics so reality is one more sample.

Tobin 2017, Peng 2018

Privileged learning / adaptation

Privileged critic or teacher; estimate what cannot be sensed.

Pinto 2018, Lee 2020, Miki 2022

Massive parallel simulation

Cheap GPU experience makes wide randomization affordable.

Rudin 2022, Isaac Lab

This project combines all four; its own choices are an asymmetric actor-critic and aggressive actuator-realism randomization.

Objectives and hypotheses

Objectives (assignment brief)

- Set up the Go2 in Isaac Lab; configure PPO / RSL-RL.
- Train walking, trotting and velocity tracking.
- Apply domain randomization (friction, mass, delay, noise).
- Validate in Isaac Sim under perturbations.
- Compare two policy configurations.
- Export to ONNX; deploy via the SDK and ROS 2 Humble.
- Analyse the sim-to-real gap and limitations.

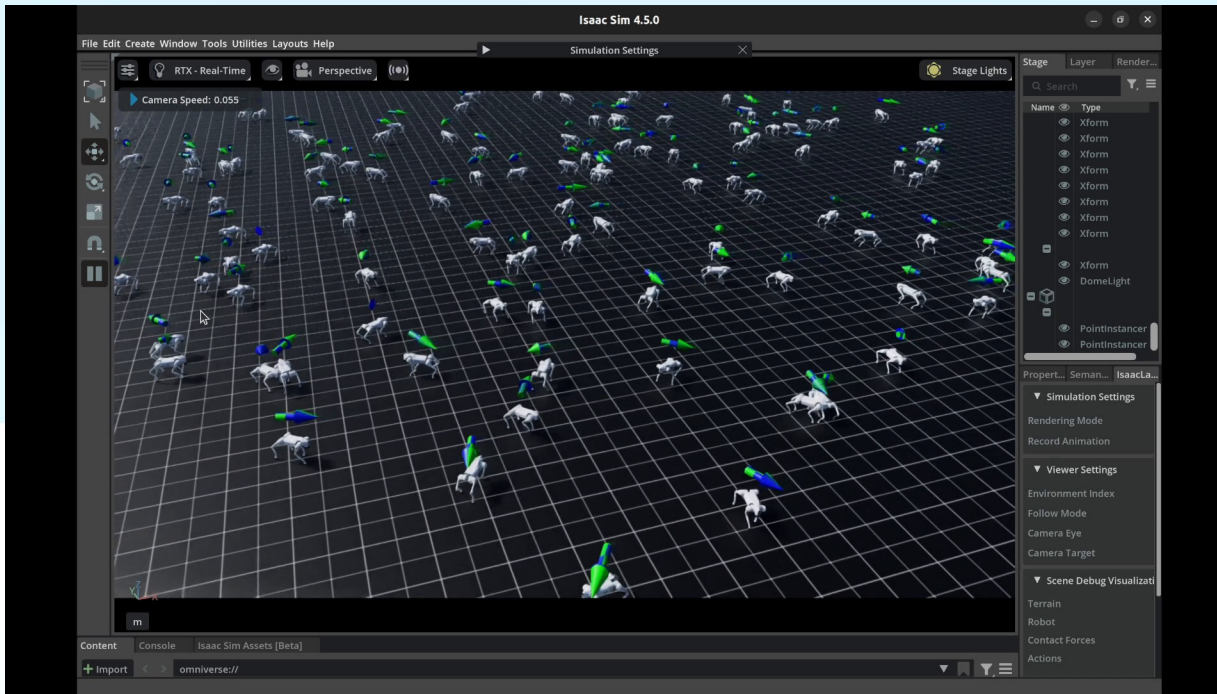
Hypotheses

- H1: in simulation the gap is invisible; all good policies look excellent.
- H2: on hardware, a policy that needs an unmeasurable input drifts; an asymmetric design walks straight.
- H3: heavier action smoothness and latency randomization remove motor chatter.

Two configurations compared

V2 symmetric (48-dim, base velocity in the actor) vs V3 asymmetric (45-dim, base velocity to the critic only).

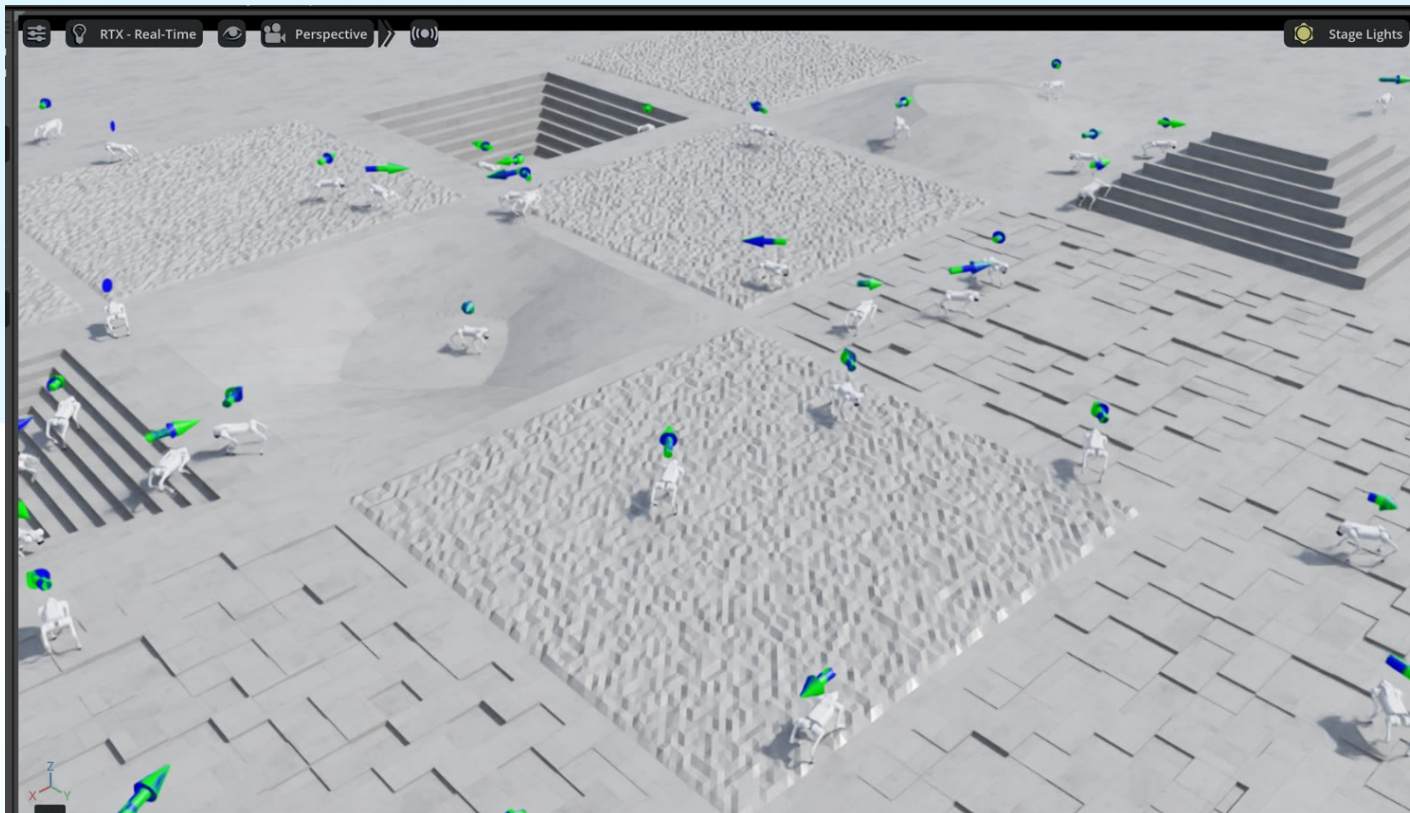
Training in Isaac Lab: flat terrain



- 16,384 robots step in parallel on one GPU.
- Reward: track command, stay upright, lift feet, save energy.
- PPO with RSL-RL and a custom reward recipe.
- A trot gait emerges; it is never hand-specified.
- 2 billion steps in 1 h 40 m on an RTX PRO 6000.

16,384 Go2 robots in parallel; arrows show each robot's commanded velocity.

Training in Isaac Lab: rough terrain



Stairs, slopes and uneven blocks; the policy adds a height-scan observation.

- Same pipeline, harder terrain: stairs and uneven ground in all directions.
- A 187-dim height scan is added to the observation.
- Trained and validated in Isaac Sim, including stair climbing.
- Hardware deployment pending: the Go2 used has no depth sensor feeding the height scan.

Inside the policy: observation, action, reward

Observation (45-dim)

- Angular velocity (IMU)
- Projected gravity
- Velocity command
- Joint positions and velocities
- Last action (memory)

Action (12)

- Joint offsets from a nominal stance
- $q = q_{\text{default}} + 0.25 a$
- 500 Hz PD loop (Kp 40, Kd 1)
- Policy at 50 Hz, about 1 ms

PPO

- Actor-critic MLP 256-256-128
- Clipped objective, GAE
- KL-adaptive learning rate
- gamma 0.99, lambda 0.95

Reward = weighted sum of 12 terms; the gait emerges from these objectives:

Positive: velocity tracking (1.5), angular tracking (0.75), foot air-time (0.5)

Negative: flat orientation (2.5), vertical velocity (2.0), action rate (0.25), undesired contacts (1.0), feet slide (0.25), joint deviation (0.1), torques and accelerations (small)

Domain randomization: the sim-to-real bridge

Every episode the simulator changes the physics, so the real robot is just one more variant.

Body mass

-2 to +4 kg

Centre of mass

+/- 5 cm

Friction (static / dynamic) 0.4-1.4 / 0.3-1.0

Actuator gains Kp / Kd +/- 20 %

Actuator latency

0-20 ms
(DelayedPDActuator)

Joint friction / armature

randomised

External force / push

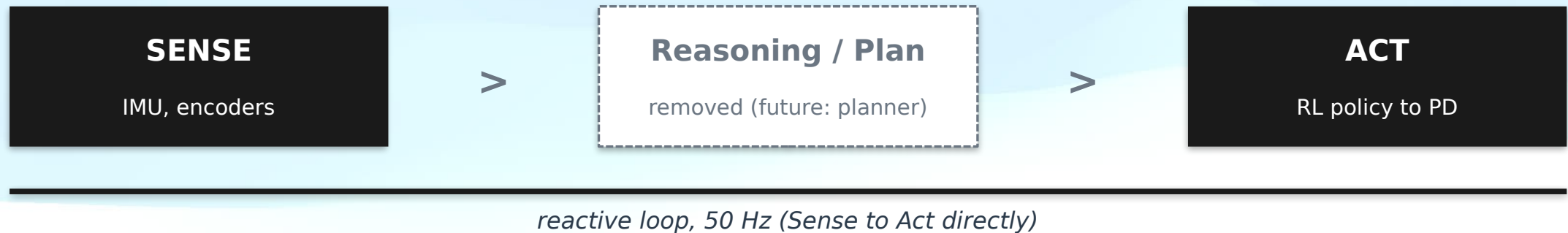
+/- 3 N, +/- 0.5 m/s

Stand-still training

20 % of robots, zero command

Where the project sits: the Sense-Plan-Act loop

The COGAR course frames every architecture on one loop. Our controller is the reactive family: Sense to Act, with the Reasoning/Plan stage removed.



- The policy is a behaviour in the course sense: a mapping of sensory inputs to motor actions (perceptual schema + motor schema).
- It is a Type-1, reactive behaviour, compiled down from a slow optimisation: exactly what reinforcement learning does.

Behaviour coordination and design patterns

Subsumption and fixed-priority coordination (Part 4)

- The deployment state machine (IDLE > STAND_UP > READY > WALK > DAMPING) is a fixed-priority, winner-take-all behaviour coordinator.
- Safety reflexes (tilt > 46 deg, sensor watchdog, NaN guard, clipping) suppress the walking behaviour, in Brooks' exact sense.
- These are reflexes in the ethological sense the course gives: rapid, automatic, involuntary responses.

Course design patterns realised (Part 2)

Adapter: the joint-order remap between Isaac Lab and the Unitree SDK conventions

Computational: the policy step: one periodic `on_update()` collecting inputs, one inference, all outputs

Publish-subscribe + Sensor-device: the IMU and encoder readers push state to the controller over DDS / ROS 2

Marr's three levels: what (a quadruped should walk) -> processes (schemas + FSM) -> implementation (ONNX on Jetson, PD on the driver).

Two-configuration comparison and metrics

V2 (symmetric) vs V3 (asymmetric)

Actor observation	48-dim (+ base vel)	45-dim (no base vel)
Privileged info	none	base vel to critic only
Actuator model	ideal	delayed 0-20 ms
Gain randomization	none	+/- 20 %
Real-robot result	drifts sideways	walks straight

Simulation metrics

Survival rate	99.7 %
Fall rate	0.33 %
Velocity tracking error	0.22 m/s
Angular tracking error	0.15 rad/s
Final mean return	~31.2

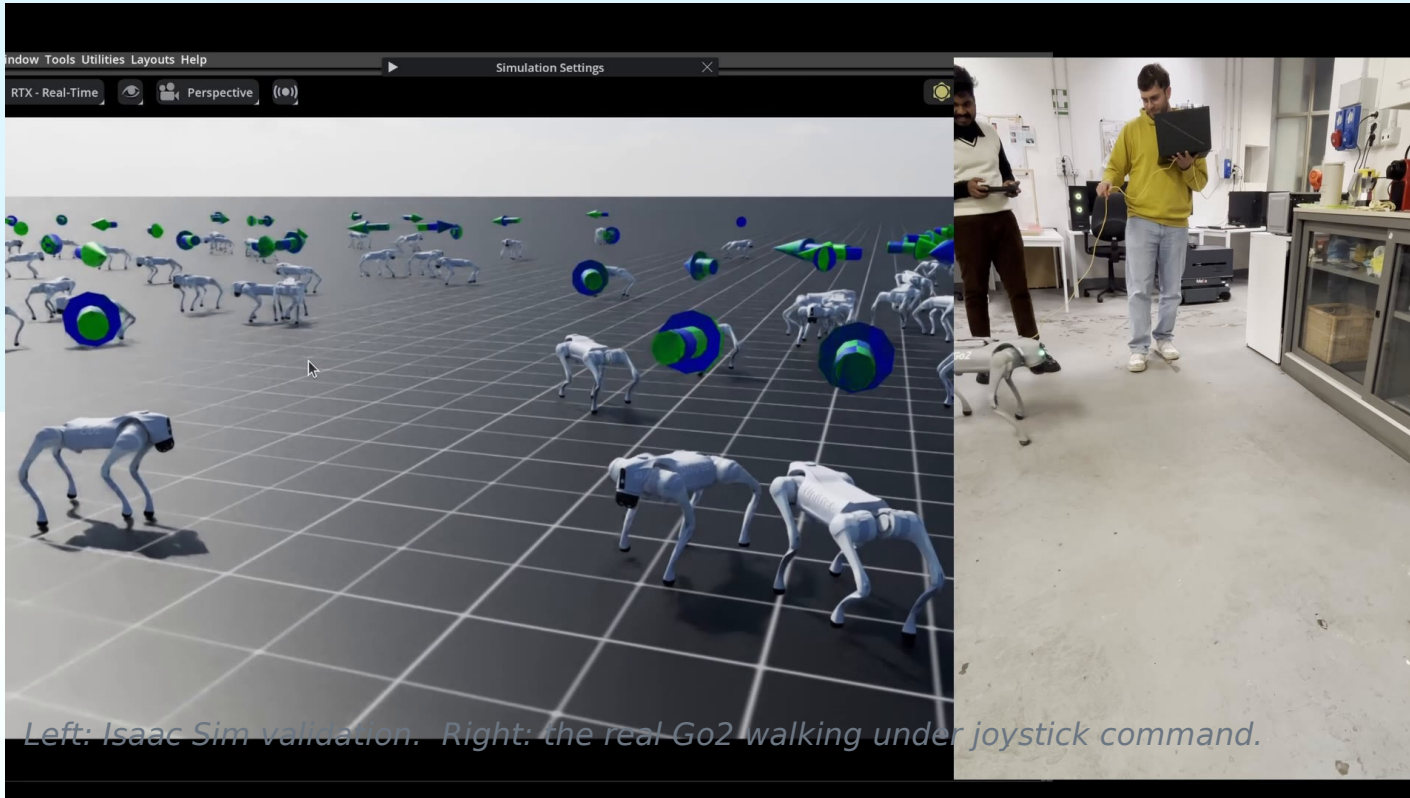
H1 confirmed: in simulation both configurations are near-perfect. The gap appears only on hardware.

The sim-to-real gap: failures and fixes

Failure	Root cause	Fix
Foot dragging	air-time reward too low	x40 air-time and feet-slide penalty
Collapsing	orientation penalty too harsh	rebalance so tracking dominates
Motor chatter	actions change too fast	x5 action-rate and accel. penalties
Sideways drift	actor used base velocity it cannot measure	asymmetric actor-critic (vel to critic only)
Gain sensitivity	real latency / gains / stiction	DelayedPDActuator and +/-20% gains
Twisted motion	sim vs SDK joint order	verified joint remap (Adapter pattern)

The asymmetric actor-critic was the single largest improvement to real-world performance.

Real-robot deployment: zero-shot on the Go2 EDU



- Same network that finished training; no fine-tuning.
- ONNX inference at 50 Hz.
- Run from a laptop over Ethernet and from the onboard Jetson Orin NX.
- The Jetson allows fully untethered operation.
- Walks straight (asymmetric actor-critic); holds station on neutral command.

What the evidence supports, honestly

- Zero-shot flat walking on the real Go2: supported (demonstration; the specific failure modes were removed).
- Asymmetric actor-critic removes drift: supported by the V2 vs V3 comparison (H2).
- Domain randomization bridges the gap: supported by robustness across the training distribution.

Honest scope

Simulation metrics are quantitative; the hardware comparison is qualitative (a logged real-robot benchmark is future work).

The system realises the reactive behaviour and its fixed-priority coordinator, not a deliberative Reasoning/Plan layer.

Rough terrain is validated in simulation only. Next: add a planner (hybrid architecture), a depth sensor, and a learned velocity estimator.

Thank you

Go2 Reinforcement Learning Locomotion and Sim-to-Real Transfer

Acknowledgments

Sincere thanks to Prof. Fulvio Mastrogiovanni and Gianluca Galvagni for the Cognitive Architectures for Robotics course and for their guidance and support throughout this project.

Demonstration video (Isaac Lab training to real deployment):

<https://drive.google.com/file/d/1e1jpoKKcbISyVp17HyGn1CQgQ9hJk3pM/view>

Richard Albert King Mechoda | ID 8525970